ADF to Fabric Migration Agent

Status report · 4 November 2026

The first page gives the current status, what comes next, estimates and blockers. Section 2 describes the current implementation: what has been built, how the tool is designed, how a migration is carried out and checked, and how far each part is done. Figures are as of 4 November 2026. The terms used are explained at the end.

Phase 1: what we did (9 to 16 October 2026)

Phase 1 built the core path: read an ADF factory, convert its pipelines, and deploy them to Fabric.

Area	What we did
Reading ADF	Read the factory from a GitHub or Git repository, a ZIP file, the live factory in Azure, or its published export
Converting	Converted pipelines, their steps and their schedules into Fabric's format
Deploying	Published the converted pipelines to a Fabric workspace, including pipelines that call other pipelines
Web UI	A first web UI to run the migration, with sign-in to Azure and Fabric
Compute	An estimate of the Fabric capacity each pipeline needs, and the cost of keeping Databricks
Notebooks	Ran the Databricks notebooks of `PL_Spark_Test` as Fabric notebooks, and flagged notebook code that reads storage directly
Storage	Created OneLake shortcuts, so Fabric can read existing Azure storage without copying it
Data warehouse	Assessed the Synapse data warehouse that would move to a Fabric Warehouse

Phase 2: what we implemented next (18 October to 4 November 2026)

Phase 2 made the migration complete, checked and safe to use, and proved it on the sample factory.

Area	What we implemented
Connections	Checked the list of supported stores against Microsoft's; separate sign-ins for ADF and Fabric; the option to skip a connection
Nothing missed	Every step of every pipeline is recorded and checked by a second scan; missing connections and datasets are reported with who must supply them
Order of migration	Worked out which pipelines depend on which, and the order to move them in
Early warnings	Problems the tool cannot fix are listed before converting, including the 5,000-row Lookup limit
Agreeing the plan	A few questions with recommended answers, saved as the plan that drives the run
Notebook migration	Read, check, convert, deploy and track Databricks notebooks; check and link a deployed notebook from the Deploy step
Fabric set-up	List what the workspace needs, show what exists, create only what a person approves; Spark pool and environment created for real
Fabric storage	Map Azure storage to Fabric, and optionally read CSV and Parquet data through Fabric's own storage
Run and compare	Run each pipeline in Fabric and compare every step with its ADF run; start runs from the web UI
Final checks	Four independent checks that decide whether a migration is **Verified**

Area	What we implemented
No reference run	Check the data a pipeline wrote when there is no ADF run to compare with
Reports	Side-by-side view of each ADF and Fabric pipeline, a problem finder that ranks fixes, and a confidence range per pipeline
Save to Git	Save the migration's items to the workspace's Git repository
Security	Roles, separation between organisations, audit trail, and the fixes from a security review
Web UI	Nine guided steps, typed confirmations, saved progress, and clicks cut from 4.6 to 0.4 seconds
Sample migration	Moved the sample to the adf2fabricmigration workspace, handled the Lookup limit, and ran all 10 pipelines with 0 differences

1 Current status

The sample ADF factory `adf-data-migration-7593` has been migrated to the Fabric workspace **adf2fabricmigration**. All 10 pipelines are converted, deployed and run. All 10 Fabric runs of 4 November match their reference run: 132 checks, 0 differences. One step is left before the migration is marked **Verified**: linking the two notebooks to the pipeline that runs them.

Measure	Result
Pipelines converted	10 of 10
Activities converted	56 of 56, all automatically; none needed manual work
Datasets	14 used and carried over; 4 unused, left in ADF
Connections	4 (3 to Azure Data Lake Storage, 1 to Databricks), all of a type the tool can create in Fabric
Pipelines deployed to adf2fabricmigration	10 of 10
Pipelines run and compared	10 of 10 match: 8 against their ADF run, 2 against an earlier Fabric run
Final checks	3 of 4 pass; the conversion check waits on the 2 notebooks

Results by pipeline

Pipeline	Steps	What it does	Compared with	Checks matched
`PL_Compute_Test`	4	Sets a value, waits, reads customers, copies a CSV file to Parquet	ADF run, 15 Oct	14
`PL_ControlFlow_Test`	5	Loops over tables and branches on a condition	ADF run, 21 Oct	10
`PL_Copy_Test`	1	Copies one file	ADF run, 21 Oct	6
`PL_Generate_CSV_And_Parquet`	4	Copies customer and sales data to CSV and Parquet	ADF run, 21 Oct	20
`PL_Rig_Orchestrator`	19	Lists the CSV files in a folder and runs the child pipeline for each one	Fabric run, 3 Nov	30
`PL_Rig_Child_FileToParquet`	5	Converts one CSV file to Parquet and checks the row count	Fabric run, 3 Nov	14
`PL_Serverless_Test`	5	Reads Parquet files and fails if they are empty	ADF run, 30 Oct	12

Pipeline	Steps	What it does	Compared with	Checks matched
`PL_Spark_Test`	2	Runs 2 notebooks, now as Fabric notebooks instead of Databricks	ADF run, 16 Oct	4
`PL_SQL_Object_Test`	5	Lists folders in storage and checks there are enough	ADF run, 30 Oct	8
`PL_Validation`	6	Compares the row counts of CSV and Parquet copies	ADF run, 30 Oct	14

The two `PL_Rig` pipelines are compared with an earlier Fabric run because of the 5,000-row Lookup limit (section 2.7). The child pipeline runs 4 times per orchestrator run, once per CSV file, and all 4 runs are compared.

What is still open on the sample

`PL_Spark_Test` runs two notebooks, `NB_Read_Raw_Data` and `NB_Transform_Cab_Data`. Both are converted, deployed to adf2fabricmigration and run successfully. Each must still be linked to its record on the Deploy step with the **Bind** button. The tool has already confirmed that the code in Fabric is the same as the converted code. The link records that a person has accepted it.

1.1 What is next

1. **Finish the sample.** Link the two notebooks on the Deploy step, run the final checks again, and confirm the status is **Verified**.
2. **Make notebook conversion safe to repeat.** Today, converting a notebook again when its code is missing wipes the good result. It should keep the good result and report the missing code.
3. **Output check in the web UI.** Checking the data a pipeline wrote (used when there is no reference run) works only from the command line today. Add it to the Report step.
4. **Walk through the web UI as a user.** Go through all 9 steps on the demo project and on the sample. Note where a user gets stuck, where a button is disabled with no reason, and where a step is slow, and fix them.
5. **Bring command-line features into the web UI.** Notebook conversion and deployment, OneLake shortcuts, Spark pool set-up, and the problem diagnosis.
6. **Cover more of ADF.** Create more connection types automatically, convert event-based triggers, change data capture (CDC) and mapping data flows.
7. **Prove the remaining Fabric set-up on a real system.** Creating a workspace, capacity, user access and a Lakehouse, and saving the migration to a Git repository from Fabric.

1.2 Estimates

These are rough estimates for one developer, in working days, including testing and documentation. Review time and the time spent waiting on the blockers in 1.4 are not included.

Work item	Estimate	What it involves
Safe notebook conversion	1 day	Keep the existing result when the code is missing, and a test for it
Output check in the web UI	2 to 3 days	A new screen on the Report step, with permissions and tests
Web UI walkthrough and fixes	3 to 5 days	Walk 9 steps on 2 projects, then fix what is found
Command-line features in the web UI	3 to 5 days	About 5 new screens, each with permissions and tests

Work item	Estimate	What it involves
More connection types	1 to 2 days per type	Each new type is tried against Fabric before it is relied on
Event-based triggers	3 to 5 days	Triggers that start a pipeline when a file arrives or an event is raised
Change data capture (CDC)	3 to 5 days	Turning ADF change-capture into Fabric copy jobs
Remaining Fabric set-up on a real system	2 to 3 days, once access is in place	Run the existing set-up features for real and fix what Fabric rejects

1.3 Blockers

Three blockers, in priority order. They come down to one sign-off and two kinds of access.

Priority	Blocker	What it holds up	What is needed
1. High	**The notebooks are not linked yet.** The tool has checked them, but linking records a person's acceptance, so the tool does not do it itself	The **Verified** status for the sample, and so the sign-off of the sample migration	A person with the approver role clicks **Bind** for both notebooks on the Deploy step
2. Medium	**No Fabric admin access for testing.** We cannot create workspaces or capacity, and no workspace is linked to a Git repository	Proving on a real system that the tool can set up a full Fabric workspace and save the migration to Git	Admin rights in a test environment, and one test workspace linked to a Git repository

2. Current implementation

2.1 Purpose

Many data teams run their pipelines in Azure Data Factory and want to move them to Microsoft Fabric. Doing this by hand means rewriting each pipeline, recreating its connections, and then proving that the new pipeline behaves the same as the old one.

The tool does that work. It reads the existing ADF setup, converts each pipeline for Fabric, deploys it to a Fabric workspace, runs it, and proves the result. It never changes anything in ADF. It writes nothing to Fabric until a person has seen a preview and confirmed it.

2.2 Work completed so far

Everything below is built and covered by automated tests. "Tried for real" means it has also been run against the real ADF factory or Fabric workspace.

Reading the ADF setup

- **Four ways to read it.** From a Git repository, a ZIP file, the live factory in Azure, or the factory's published export. Tried for real.
- **Nothing missed.** Every step of every pipeline is recorded, and a second, independent scan confirms none was skipped. Anything that cannot be handled is listed with the reason.
- **Order of migration.** The tool works out which pipelines, datasets and connections depend on which, and suggests an order to move them in, shown as waves.
- **Early warnings.** Before converting, the tool lists problems it cannot fix itself: missing connections or datasets (and who needs to supply them), connection settings with no value, and Lookup steps that may hit the 5,000-row limit.

Agreeing the plan

- **A few clear questions.** The tool asks only what matters for the run (which pipelines, which workspace, storage and compute), each with a recommended answer.
- **Tied to one workspace.** A plan agreed for one Fabric workspace cannot be used to deploy to another.
- **One person by default.** No second approver is needed in this phase; a mode with a second approver exists for later.

Converting pipelines

- **41 step types.** Each has a conversion rule based on Microsoft's published definitions. A step type with no rule is shown as a blocker, never dropped silently.
- **Expressions and datasets.** ADF expressions are rewritten for Fabric. Fabric has no separate datasets, so each dataset's settings are copied into the steps that use it, and every dataset is accounted for.
- **Pipelines that call pipelines.** ADF's "Execute pipeline" becomes Fabric's "Invoke pipeline", pointing at the converted child.
- **Databricks steps.** These either keep running on Databricks with the same cluster settings, or run the notebook in Fabric instead.
- **Schedules.** Scheduled triggers are converted and created switched off, so nothing runs by surprise.
- **Fabric storage.** Optionally, CSV and Parquet data in Azure storage is read through Fabric's own storage instead. Tried for real.

Connections and storage

- **A decision for every connection.** The list of supported stores was checked against Microsoft's. Each ADF connection gets a decision, and a person can choose to skip one.
- **Created automatically for 5 types.** Azure SQL and Synapse, Azure Data Lake Storage, Blob storage, web (REST) services and Databricks. The sample's storage connection is used by its real runs.
- **OneLake shortcuts.** The tool maps Azure storage folders to Fabric and creates the shortcuts. Tried for real.

Notebooks, compute and Fabric set-up

- **Cost and size.** Each pipeline gets an estimate of the Fabric capacity it needs.
- **Notebook conversion.** Databricks notebooks are read (only with permission), checked, rewritten for Fabric, deployed and tracked when they run. Tried for real with the two sample notebooks.
- **Notebook check and link.** On the Deploy step, a person checks that the notebook in Fabric matches the converted code, then links it.
- **Spark pool sizing.** A Fabric Spark pool is proposed to match each Databricks cluster.
- **Fabric set-up.** The tool lists what the workspace needs, shows what already exists, and creates only the items a person approves. The Spark pool `ADF-Small-n1` and its environment were created this way for real.
- **Data warehouse moves.** Built to move a Synapse data warehouse to a Fabric Warehouse. Not yet tried for real, because the source warehouse was paused.

Deploying

- **Deploy to Fabric.** Creates or updates each pipeline in the workspace, after a preview. It refuses to deploy a pipeline that still points at a notebook or job that does not exist yet. Tried for real.
- **A record of where.** Each deployment is recorded with its workspace, so a pipeline deployed elsewhere is not counted as deployed here.
- **Notebook deploy.** Publishes a converted notebook to the workspace. Tried for real.
- **Save to Git.** Saves only the migration's own items to the workspace's Git repository. Built; not yet tried for real.

Running and checking

- **Run and compare.** Runs each pipeline in Fabric and compares every step with the last successful ADF run: whether it succeeded, rows read and written, and what lookups returned. A run made before the latest deployment is treated as out of date. Tried for real.
- **Runs from the web UI.** The Report step can start runs of up to 20 named pipelines after a typed confirmation, and records who started them.
- **Final checks.** Four independent checks (section 2.6) decide the overall status. Only **Verified** counts as complete. Tried for real.
- **Output check.** For a pipeline with no reference run, compares the data it wrote with the expected data. Tried for real on Fabric storage.
- **Side-by-side report.** Shows each ADF pipeline next to its deployed Fabric version: steps, order, calls, data and connections.
- **Problem finder.** For each pipeline, names the first thing stopping it and ranks the fixes by how many pipelines each one would unblock. Also gives a confidence range and a readiness level for handing each pipeline over.
- **AI-drafted fixes (off by default).** For a pipeline the tool could not convert, an AI model can suggest corrected ADF definitions. A person must accept the suggestion, and the tool's own rules still produce the Fabric pipeline.

Security

- **Roles.** Five roles (viewer, analyst, approver, operator, admin). Each action needs the right role, and the server checks it.
- **Separation.** Each organisation sees only its own projects.

- **Protection.** Sign-in locks after repeated failures, Git addresses are checked before use, and a request sent twice cannot start the same job twice.
- **Audit trail.** Every change is logged, and the log can be checked for tampering.
- **Security review.** Issues found in a security review were fixed, and every third-party package is pinned to a known version.
- **Reliability.** Saved records cannot be half-written, and finished jobs survive a server restart.

Web UI

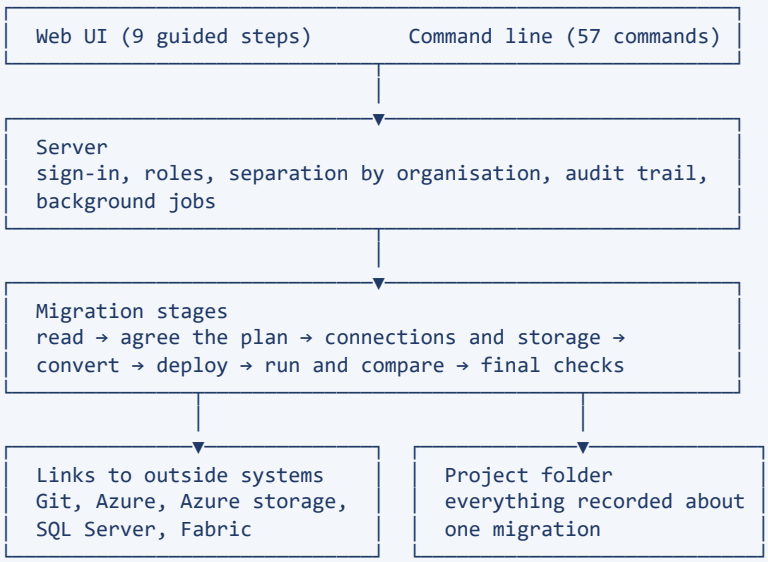
- **Guided steps.** Nine steps with a progress list. Every change to Fabric needs a typed confirmation. Progress is saved, so a page reload loses nothing.
- **Two sign-ins.** ADF and Fabric can be reached with different accounts at the same time.
- **Review screens.** Migration waves and order, compute, compatibility, and option cards pre-filled from the agreed plan.
- **Speed.** A click went from 4.6 seconds to 0.4 seconds. A loading indicator shows while work runs.
- **Accessibility.** Text and colour contrast meet the common accessibility standard.
- **Demo.** A built-in sample project for demonstrations.

Testing

- **2,283 automated tests.** They run in about 4.5 minutes without touching any real Azure or Fabric system.
- **Real-system checks.** Every new kind of change to Fabric was also tried once on a real workspace, with something safe to remove.

2.3 Architecture

Layers



What each layer does

- **Web UI and command line.** Two ways to run the same stages. The web UI only sends requests to the server; all the work happens behind it. The command line suits scripted or bulk work.
- **Server.** Checks who the user is and what their role allows, keeps each organisation's projects apart, and logs every change. Any change to Fabric needs a preview first and then a confirmation. Internal details such as file paths and tokens never reach the browser.

- **Migration stages.** Each stage does one job, reads what the previous stage saved, and saves its own result, so every step can be inspected and repeated.
- **Links to outside systems.** Read access to ADF and Azure storage, and write access to Fabric, used only after a confirmation.
- **Project folder.** Everything about one migration is kept in one folder. Passwords and keys are never stored there.

Design rules

- **Nothing is lost silently.** Everything the tool reads is either converted or listed with a reason.
- **No guessing.** The tool never invents a connection or a Fabric setting. What it cannot decide, it asks a person.
- **Fixed rules, not AI.** The Fabric pipelines are produced by fixed conversion rules, so the same input always gives the same output.
- **Honest status.** Fabric accepting a pipeline is not proof that it works. Only a real run, checked independently, can mark a migration as **Verified**.

2.4 How a migration works

#	Stage	What happens
1	Read	The ADF setup is read from Git, a ZIP file, the live factory or its published export, and checked for missing parts
2	Discover	Every pipeline, step, dataset, connection and trigger is recorded, with the order to migrate them in
3	Agree the plan	A few questions (which pipelines, which workspace, storage, compute), each with a recommended answer
4	Connections and storage	Each ADF connection gets a Fabric connection; storage folders get OneLake shortcuts where chosen
5	Convert	Each pipeline is rewritten for Fabric
6	Deploy	Pipelines and notebooks are published to the workspace after a preview and a confirmation; schedules start switched off
7	Run and compare	Pipelines run in Fabric and are compared with their ADF runs
8	Final checks	Four independent checks decide the overall status (section 2.6)

Notebooks and compute

`PL_Spark_Test` ran two notebooks on a Databricks cluster. The tool rewrote both notebooks for Fabric and proposed a matching Fabric Spark pool, `ADF-Small-n1` (small, 4 cores, 1 node). It created the pool and its environment in the workspace, deployed the notebooks, and the pipeline now runs them in Fabric.

On the Deploy step, each notebook has a box with its Fabric id already filled in. **Check** compares the notebook in Fabric with the converted code. **Bind** links it once they match. Neither button changes anything in Fabric.

2.5 Navigating the web UI

First, the person signs in, chooses the Fabric account and workspace, clicks **Test connection**, then **Start migration**. The nine steps are listed on the left; each says what it needs from the person and is ticked when done.

Step	What the person does	What it produces
1. Discover	Point the tool at the ADF setup	A record of everything found
2. Review	Look over pipelines, steps, dependencies, migration waves, compute and compatibility	A reviewed list
3. Consult	Confirm or change the recommended answers	The agreed plan

Step	What the person does	What it produces
4. Connect	Choose how each ADF connection is set up in Fabric, and test it	A decision for every connection
5. Plan target	See what the workspace needs and what already exists	A set-up plan
6. Convert	Convert the pipelines for Fabric	Fabric pipelines and a conversion report
7. Validate	Check the converted pipelines against ADF	A result for each pipeline
8. Deploy	Preview, type the confirmation, publish; check and link the notebooks	Pipelines in the workspace
9. Report	Run and compare the pipelines, and read the final status	A status for each pipeline and overall

2.6 How we validate the pipelines

Four final checks

The tool does not take the conversion's word for it. It reads ADF and the converted pipelines again and makes four checks:

Check	Question it answers	Sample result
Conversion	Does each Fabric pipeline cover every step, connection and notebook of the ADF pipeline?	Waits on the 2 notebook links
Fabric acceptance	Does Fabric accept the pipeline?	Pass, 10 of 10
Deployment	Is the pipeline in the chosen workspace, exactly as converted?	Pass, 10 of 10
Run	Has it run in Fabric since it was last deployed, and did it behave like the original?	Pass, 10 of 10

The migration is **Verified** only when all four pass. Fabric accepting a pipeline is never reported as a working migration.

Run and compare

For the run check, each pipeline is run in Fabric and compared, step by step, with its last successful ADF run:

- did each step succeed or fail,
- how many rows each copy read and wrote,
- what each lookup returned.

On the sample, that was 132 checks across the 10 pipelines, with 0 differences. Only counts and fingerprints are kept, never the data itself. A run made before the latest deployment is out of date and must be repeated.

When there is no ADF run to compare with

A new client may never have run a pipeline in ADF, or its runs may not have moved any data. Two options cover this, and neither needs the client to load test data and clean it up later:

- **Compare with an earlier Fabric run.** Once one Fabric run has been checked, later runs are compared with it. The two `PL_Rig` pipelines are checked this way.
- **Check the output.** The tool reads the files the pipeline wrote and compares the row count, the column names and a fingerprint of the contents with the expected output. The report names the outputs that were checked.

2.7 Handling the Lookup limit

The problem

`PL_Rig_Orchestrator` lists the CSV files in a folder and runs `PL_Rig_Child_FileToParquet` once per file. The child pipeline checks its own work:

1. A Lookup step reads the CSV file to count its rows.
2. A Copy step converts the file to Parquet.
3. A condition compares the two counts and fails the run if they differ.

A Lookup step returns at most 5,000 rows, in ADF and in Fabric alike. Once a file grew past 5,000 rows, the count was cut short and the check failed, even though the copy was correct. The ADF pipeline fails the same way on today's data.

How we handled it

- **ADF was left unchanged.** The pipeline belongs to the client, so we did not edit it.
- **The Fabric pipeline uses a count with no limit.** The Copy step already reports how many rows it read and how many it wrote. The Fabric version compares those two numbers instead of the Lookup count. The check means the same thing, and it works for any file size.
- **A fair comparison.** Because the ADF pipeline fails on today's data, it cannot be the reference. Both `PL_Rig` pipelines are compared with their successful Fabric runs of 3 November, and the report says so.
- **Warning for every project.** The tool now warns about any Lookup step that may hit the 5,000-row limit, before conversion.

2.8 How far each part is done

Area	Status	Detail
Reading the ADF setup	Done	All four ways in
Converting pipelines	Done	41 step types; SSIS packages, U-SQL and classic Azure ML have no Fabric equivalent and are flagged
Datasets	Done	Copied into the steps that use them
Deploying to Fabric	Done, tried for real	Preview first; schedules start switched off
Run and compare	Done, tried for real	Step results, row counts, lookup results
Final checks	Done, tried for real	Four checks, one overall status
Checking without a reference run	Done, command line only	CSV and Parquet output
Notebooks	Done for Databricks notebooks	Convert, deploy, check and link; custom code libraries not handled
Connections	Partly done	5 types created automatically; about 50 others get a decision and are set up by hand
Triggers	Partly done	Schedules convert; event-based triggers are set up by hand
Storage	Partly done	Azure Data Lake Storage through OneLake shortcuts; CSV and Parquet only
Fabric set-up	Partly done	Spark pool and environment created for real; workspace, capacity, user access and Lakehouse not yet tried for real
Save to Git	Built, not tried for real	No Git-linked workspace yet

Area	Status	Detail
Mapping data flows	Found and reported only	The step that runs it becomes a step that runs a Fabric dataflow; the dataflow itself is built by hand
Change data capture (CDC)	Found and reported only	Not converted

3. What we plan to do next

The plan runs in four phases. The timings come from the estimates in 1.2, for one developer.

3.1 Complete the sample (this week)

- Link the two notebooks on the Deploy step, run the final checks again, and mark the sample migration **Verified**.
- Make notebook conversion safe to repeat, so a good result can never be wiped.

3.2 Make the web UI complete (next 3 to 4 weeks)

- Add the output check to the Report step, so a pipeline with no reference run can be proven from the web UI.
- Walk through all 9 steps as a user, on the demo and on the sample, and fix where a user gets stuck.
- Bring the command-line features into the web UI: notebook conversion and deployment, OneLake shortcuts, Spark pool set-up and the problem finder.

3.3 Cover more of ADF

- Create more connection types in Fabric automatically.
- Convert event-based triggers.
- Convert change data capture (CDC) to Fabric copy jobs.
- Rebuild mapping data flows in Fabric.

3.4 As soon as admin access is granted

- Prove on a real system that the tool can set up a full Fabric workspace (workspace, capacity, user access and Lakehouse) and save the migration to a Git repository.